

CS-4053 **Recommender System**

History-based Techniques

- ❑ Use historical data to make recommendations
 - ❑ Collaborative Filtering
 - ❑ Content-based Filtering
- ❑ These techniques require data (*a lot of it!*)
- ❑ Are not adaptable
- ❑ Do not *ask* user for preferences

Knowledge-based Recommender System

- ☐ Knowledge-based recommender systems exploit user requirements
- ☐ These systems are appropriate when:
 - ☐ Customers want to explicitly specify their requirements
 - ☐ It is difficult to obtain ratings for a specific type of item
 - ☐ When ratings are time-sensitive

Knowledge-based Recommender System

- ❑ Application domain
 - ❑ Expensive items, not frequently purchased, few ratings (car, house)
 - ❑ Time span important (technological products)
 - ❑ Explicit requirements of user (buying property)

Conceptual Goals of Recommender Systems

Approach	Conceptual Goal	Input
Collaborative	Give me recommendations based on a collaborative approach that leverages the ratings and actions of my peers/myself.	User ratings + community ratings
Content-based	Give me recommendations based on the content (attributes) I have favored in my past ratings and actions.	User ratings + item attributes
Knowledge-based	Give me recommendations based on my explicit specification of the kind of content (attributes) I want.	User specification + item attributes + domain knowledge

Knowledge-based Systems: **Example**

- ☐ If you're looking for a house or a car online. You input price, how many rooms, how much total floor space, etc., and the website returns a list of houses based on those constraints
- ☐ **Issue:**
 - ☐ Is this recommendation or simple query search?

Knowledge-based Recommender System

❑ Knowledge-based systems can be categorized into

❑ **Constraint-based Systems**

- Explicitly defined conditions

❑ **Rule-based Systems**

- Similarity to specified requirements

Constraint-based Systems

- ☐ The user specifies his or her initial preferences
 - ☐ All at once
 - ☐ Incrementally using a wizard
 - ☐ Interactive dialog
- ☐ The user is presented with a set of matching items
 - ☐ With an explanation as to why a certain item was recommended

Components of Constraint-based Systems

- ☐ Customer properties
- ☐ Product properties
- ☐ Constraints
- ☐ Filter conditions
and product
- ☐ Products

V_C

V_{PROD}

C_R (on customer properties)

C_F – relationship between customer

C_{PROD} – possible instantiations

Knowledge base: **Example**

$V_C = \{$
 kl_c : [expert, average, beginner] /* level of expertise */
 wr_c : [low, medium, high] /* willingness to take risks */
 id_c : [shortterm, mediumterm, longterm] /* duration of investment */
 aw_c : [yes, no] /* advisory wanted ? */
 ds_c : [savings, bonds, stockfunds, singleshare] /* direct product search */
 sl_c : [savings, bonds] /* type of low-risk investment */
 av_c : [yes, no] /* availability of funds */
 sh_c : [stockfunds, singleshare] /* type of high-risk investment */
 }

$V_{PROD} = \{$
 $name_p$: [text] /* name of the product */
 er_p : [1..40] /* expected return rate */
 ri_p : [low, medium, high] /* risk level */
 $mniv_p$: [1..14] /* minimum investment period of product in years */
 $inst_p$: [text] /* financial institute */
 }

Knowledge base: **Example**

$C_R = \{CR_1: wr_c = high \rightarrow id_c \neq shortterm,$
 $CR_2: kl_c = beginner \rightarrow wr_c \neq high\}$

$C_F = \{CF_1: id_c = shortterm \rightarrow mniv_p < 3,$
 $CF_2: id_c = mediumterm \rightarrow mniv_p \geq 3 \wedge mniv_p < 6,$
 $CF_3: id_c = longterm \rightarrow mniv_p \geq 6,$
 $CF_4: wr_c = low \rightarrow ri_p = low,$
 $CF_5: wr_c = medium \rightarrow ri_p = low \vee ri_p = medium,$
 $CF_6: wr_c = high \rightarrow ri_p = low \vee ri_p = medium \vee ri_p = high,$
 $CF_7: kl_c = beginner \rightarrow ri_p \neq high,$
 $CF_8: sl_c = savings \rightarrow name_p = savings,$
 $CF_9: sl_c = bonds \rightarrow name_p = bonds \}$

$C_{PROD} = \{CPROD_1: name_p = savings \wedge er_p = 3 \wedge ri_p = low \wedge mniv_p = 1 \wedge inst_p = A;$
 $CPROD_2: name_p = bonds \wedge er_p = 5 \wedge ri_p = medium \wedge mniv_p = 5 \wedge inst_p = B;$
 $CPROD_3: name_p = equity \wedge er_p = 9 \wedge ri_p = high \wedge mniv_p = 10 \wedge inst_p = B\}$

Constraint-based Systems: **Example**

- ❑ Consider a simple food recommendation system with a knowledge-based containing attributes and constraints for both restaurant and customer
- ❑ The knowledge base presented is a minimal representation

Constraint-based Systems: Example

$V_c =$

```
{  
   $v_{c1}$  : [vegan, no_dairy, no_requirements] ..... /* dietary requirements */  
   $v_{c2}$  : [fast_food, traditional, continental, none] ..... /* food preference */  
   $v_{c3}$  : [high, medium, low] ..... /* spending style */  
  ...  
}
```

Constraint-based Systems: Example

$V_{\text{PROD}} =$

{

$V_{P1} : [\text{pizza, BBQ, burger, biryani, italian}] \dots \text{ /* type of food */}$

$V_{P2} : [300, \dots, 4000] \dots \text{ /* food price */}$

$V_{P3} : [\text{plaintext}] \dots \text{ /* name of food */}$

...

}

Constraint-based Systems: Example

$C_R =$

{

$C_{R1} : v_{C1} = \text{vegan} \rightarrow v_{C2} \neq \text{fast food}$

$C_{R2} : v_{C2} = \text{continental} \rightarrow v_{C3} \neq \text{low}$

...

}

Constraint-based Systems: Example

$C_F =$

{

$C_{F1} : v_{C1} = \text{vegan} \rightarrow v_{P1} \neq \text{BBQ} \wedge v_{P1} \neq \text{burger}$

$C_{F2} : v_{C3} = \text{low} \rightarrow v_{P2} < 1000$

...

}

Constraint-based Systems: Example

$C_{PROD} =$

{

$C_{PROD1} : v_{P3} = \text{afghani tikka pizza} \wedge v_{P2} = 1300$

$C_{PROD2} : v_{P3} = \text{chicken burger} \wedge v_{P2} = 750$

$C_{PROD3} : v_{P3} = \text{chicken biryani} \wedge v_{P2} = 400$

$C_{PROD4} : v_{P3} = \text{alfredo pasta} \wedge v_{P2} = 1200$

...

}

Constraint-based Systems: **Example**

❑ Now if the user sets their preference as:
{No dairy, none, low}

❑ The items recommended should be:
{chicken burger, chicken biryani}

Constraint-based Systems: **Example**

- ❑ Constraint-based recommendations can be seen as a subset selection problem
- ❑ Different techniques can be used to find subsets:
 - ❑ QuickXPlain
 - ❑ Linear Programming
 - ❑ Gradient Descent (*terrible idea!*)

Constraint-based Systems: **Issues**

- ❑ At times, users are presented with zero recommendations i.e., when no matching item is found
 - ❑ **Solution:** Relax some constraints
- ❑ Customers may be unsure about the values of their attributes
 - ❑ **Solution:** Suggest reasonable default values for attributes
- ❑ If business rules or items change often, knowledge-base has to be updated as well (*which can be an expansive task!*)

Case-based Systems

- ❑ In case-based systems, items are retrieved using similarity measures
- ❑ $\text{sim}(\mathbf{p}, \mathbf{r})$ expresses for each item attribute value, its distance to the customer requirement $\mathbf{r} \in \mathbf{REQ}$

$$\text{sim}(\mathbf{p}, \mathbf{REQ}) = \frac{\sum_{\mathbf{r} \in \mathbf{REQ}} w_{\mathbf{r}} * \text{sim}(\mathbf{p}, \mathbf{r})}{\sum_{\mathbf{r} \in \mathbf{REQ}} w_{\mathbf{r}}}$$

- ❑ $w_{\mathbf{r}}$ is the importance weight for requirement $\mathbf{r}$

Case-based Systems: Example

- ❑ Consider an example of a camera recommendation system

Item (Camera)	Price (in thousands)	Megapixel	LCD size	Waterproof
C1	12	8	3	No
C2	25	48	4	No
C3	55	60	6	Yes

- ❑ All of these available items are ***cases***

Case-based Systems: Example

- The allowed values of each attribute is given. We can use these values to normalize all the attributes

Item (Camera)	Price (in thousands)	Megapixel	LCD size	Waterproof
C1	12	8	3	No
C2	25	48	4	No
C3	55	60	6	Yes

- **Price :** { 12, 25, 55 }
- **Megapixel :** { 8, 10, 48, 54, 60 }
- **LCD size :** { 3, 4, 5, 6 }
- **Waterproof :** { Yes, No }

Case-based Systems: Example

- The allowed values of each attribute is given. We can use these values to normalize all the attributes

Item (Camera)	Price (in thousands)	Megapixel	LCD size	Waterproof
C1	0.218	0.133	0.428	0
C2	0.454	0.8	0.571	0
C3	1	1	0.857	1

- **Price :** { 12, 25, 55 }
- **Megapixel :** { 8, 10, 48, 54, 60 }
- **LCD size :** { 3, 4, 5, 6, 7 }
- **Waterproof :** { Yes, No }

Case-based Systems: Example

- ❑ Now if the user comes with the following query:

Price (in thousands)	Megapixel	LCD size	Waterproof
20	40	5	No

- ❑ We first normalize the attributes and then find similarity between each attribute in this query and the corresponding attribute in every available product

Case-based Systems: Example

- ❑ After normalizing user attributes (preferences):

Price (in thousands)	Megapixel	LCD size	Waterproof
0.363	0.666	0.714	0

- ❑ We first normalize the attributes and then find similarity between each attribute in this query and the corresponding attribute in every available product

Case-based Systems: Example

$$\text{Sim}(\text{Price}_{REQ}, \text{Price}C_1) = 1 - |0.363 - 0.218| = \mathbf{0.855}$$

$$\text{Sim}(\text{Megapixel}_{REQ}, \text{Megapixel}C_1) = 1 - |0.666 - 0.133| = \mathbf{0.467}$$

$$\text{Sim}(\text{LCD}_{REQ}, \text{LCD}C_1) = 1 - |0.714 - 0.428| = \mathbf{0.714}$$

- Assume the following weights for the attributes:

☐	w_{Price}	$= 1.5$ (<i>price matters more to this user</i>)
☐	$w_{\text{Megapixel}}$	$= 1$
☐	w_{LCD}	$= 1$

Case-based Systems: Example

- Now to find similarity of user requirements with **C1**:

$$\textit{Sim}(\textit{User}_{REQ}, \textit{C1}) = \frac{1.5 * 0.855 + 1 * 0.467 + 1 * 714}{1.5 + 1 + 1} = \mathbf{0.703}$$

- We can now find similarity of user requirements with **C2** and **C3** as well and recommend to the user a camera (*or top-k cameras*) with the highest similarity score

Knowledge-based Systems: **Limitations**

- ☐ Cost of knowledge acquisition and domain expertise
- ☐ Attributes (preferences) are not always independent